Lab 1

MODULE OVERVIEW & LEARNING OBJECTIVES

Lab Module Summary

This laboratory manual provides hands-on programming activities and cognitive challenges designed to build a deep structural understanding of Java's core object-oriented principles. Students will progress from establishing *is-a* inheritance hierarchies with superclass constructors and method overriding, to declaring abstract design contracts, and finally implementing interfaces for multiple realization and polymorphic behavior.

Learning Objectives

Upon completion of this lab module, students will be able to:

- **Implement Inheritance Hierarchies:** Establish valid *is-a* relationships between superclasses and subclasses using the extends keyword, managing proper constructor chaining and execution order.
- **Execute Method Overriding & Access Control:** Override superclass methods using the @Override annotation while evaluating encapsulation trade-offs between private and protected member access.
- **Design & Implement Abstract Classes:** Construct incomplete conceptual entities using abstract classes and methods to enforce subclass method contracts and achieve dynamic binding.
- **Utilize Interfaces & Dynamic Polymorphism:** Define, realize, and process multiple interfaces (using implements) to achieve type independence and flexible polymorphic method calls.
- **Architect an Integrated Enterprise System:** Synthesize inheritance, abstract classes, and interfaces into a unified multi-tiered domain model (University Payroll System).

LAB 1: Inheritance, Constructors, & Method Overriding

1.1 Theoretical Background

Inheritance allows a new class (subclass or derived class) to extend an existing class (superclass or base class) using the extends keyword. The subclass inherits all non-private

fields and methods, promoting code reuse. An *is-a* relationship exists between the subclass and superclass (e.g., a FinalExam is a GradedActivity).
Constructor Execution Order: In an inheritance relationship, the superclass constructor ALWAYS executes before the subclass constructor. If the superclass lacks a no-arg constructor, the subclass MUST explicitly invoke super(arguments) as the very first line of its constructor.

1.2 Source Code Listings

Listing 1.1: Superclass (GradedActivity.java)

```
public class GradedActivity {
    private double score; // Numeric score

    public void setScore(double s) {
        score = s;
    }

    public double getScore() {
        return score;
    }

    public char getGrade() {
        if (score >= 90)
            return 'A';
        else if (score >= 80)
            return 'B';
        else if (score >= 70)
            return 'C';
        else if (score >= 60)
            return 'D';
        else
            return 'F';
    }
}
```

Listing 1.2: Subclass Extending Superclass (FinalExam.java)

```
public class FinalExam extends GradedActivity {
    private int numQuestions;
    private double pointsEach;
    private int numMissed;
```

```
    public FinalExam(int questions, int missed) {
        this.numQuestions = questions;
        this.numMissed = missed;
        this.pointsEach = 100.0 / questions;
        double numericScore = 100.0 - (missed * pointsEach);
        setScore(numericScore); // Inherited public method
    }

    public double getPointsEach() { return pointsEach; }
    public int getNumMissed() { return numMissed; }
}
```

Listing 1.3: Method Overriding with @Override (CurvedActivity.java)

```
public class CurvedActivity extends GradedActivity {
    private double rawScore;
    private double percentage;

    public CurvedActivity(double percent) {
        this.percentage = percent;
        this.rawScore = 0.0;
    }

    @Override
    public void setScore(double s) {
        this.rawScore = s;
        super.setScore(rawScore * percentage); // Calls superclass version
    }

    public double getRawScore() { return rawScore; }
    public double getPercentage() { return percentage; }
}
```

1.3 Student Lab Exercise 1.1 Instructions

1. Open your Java IDE (or text editor) and create the GradedActivity, FinalExam, and CurvedActivity classes.
2. Create a test program GradeDemo.java that instantiates a CurvedActivity with a curve percentage of 1.06 (6% curve) and a raw score of 87.
3. Answer the conceptual questions in your lab sheet regarding variable visibility and modifier access.

1.4 Deliverables for Lab Activity 1

To successfully complete Lab Activity 1, submit the following items in your lab sheet / repository:

1. **Source Code Files:**
    - GradedActivity.java
    - FinalExam.java
    - CurvedActivity.java
    - GradeDemo.java (Driver class instantiating CurvedActivity with 1.06 curve percentage and 87 raw score)
2. **Program Execution Artifacts:**
    - Console output screenshot/log confirming raw score (87.00), curved score (92.22), and final grade ('A').
3. **Conceptual Analysis Report:**
    - **Question:** Why does FinalExam fail to compile if it attempts to access score = numericScore; directly?
    - **Question:** How does the protected keyword alter this behavior, and what architectural risk does protected introduce to encapsulation?

LAB 2: Abstraction with Abstract Classes & Methods

2.1 Theoretical Background

An abstract class cannot be instantiated directly using the new operator. It serves as a generic superclass from which other specialized classes extend. An abstract method has no body (ends with a semicolon) and MUST be overridden by any concrete subclass.
Abstract classes represent incomplete conceptual entities in the problem domain (e.g., a generic Student or Shape) while defining a required interface/contract for all concrete subclasses.

2.2 Source Code Listings

Listing 2.1: Abstract Superclass (Student.java)

```
public abstract class Student {
    private String name;
    private String idNumber;
    private int yearAdmitted;
```

```
    public Student(String n, String id, int year) {
        this.name = n;
        this.idNumber = id;
        this.yearAdmitted = year;
    }

    public String getName() { return name; }
    public String getIdNumber() { return idNumber; }
    public int getYearAdmitted() { return yearAdmitted; }

    @Override
    public String toString() {
        return "Name: " + name + "\nID: " + idNumber + "\nYear: " + yearAdmitted;
    }

    // Abstract Method - Must be overridden by concrete subclasses!
    public abstract int getRemainingHours();
}
```

Listing 2.2: Concrete Subclass (CompSciStudent.java)

```
public class CompSciStudent extends Student {
    private final int MATH_HOURS = 20;
    private final int CS_HOURS = 40;
    private final int GEN_ED_HOURS = 60;

    private int mathHours;
    private int csHours;
    private int genEdHours;

    public CompSciStudent(String n, String id, int year) {
        super(n, id, year); // Explicit call to superclass constructor
    }

    public void setMathHours(int math) { this.mathHours = math; }
    public void setCsHours(int cs) { this.csHours = cs; }
    public void setGenEdHours(int genEd) { this.genEdHours = genEd; }

    @Override
    public int getRemainingHours() {
        int reqHours = MATH_HOURS + CS_HOURS + GEN_ED_HOURS;
```

```
        return reqHours - (mathHours + csHours + genEdHours);
    }
}
```

2.3 Student Lab Exercise 2.1 Instructions

1. Write the Student abstract class and CompSciStudent subclass.
2. In a driver program StudentDemo.java, attempt to compile the line:
   Student s = new Student("Alice", "101", 2024);
   Observe and record the compiler error message.
3. Modify the code to instantiate a CompSciStudent reference variable assigned to a Student superclass type (Polymorphism):
   Student s = new CompSciStudent("Bob", "102", 2024);
   Verify that calling s.getRemainingHours() executes dynamic binding correctly.

2.4 Deliverables for Lab Activity 2

Submit the following deliverables for Lab Activity 2:

1. **Source Code Files:**
   - Student.java (Abstract superclass)
   - CompSciStudent.java (Concrete subclass)
   - StudentDemo.java (Driver testing polymorphic assignment and dynamic binding)
2. **Compiler Exception Log:**
   - Copy of the exact Java compiler error message generated when attempting to instantiate new Student(...) directly, along with a 1-2 sentence explanation of why instantiation is prohibited.
3. **Polymorphic Execution Verification:**
   - Console output screenshot proving dynamic binding when invoking s.getRemainingHours() through the Student reference variable.

LAB 3: Interfaces, Realization, & Interface Polymorphism

3.1 Theoretical Background

An interface specifies behaviors for a class without providing implementation details. All methods in an interface are implicitly public abstract, and all fields are implicitly public static

final. A class realizes an interface using the implements keyword.
Key Advantage: While Java restricts a class to extending only ONE superclass (single inheritance), a class may implement multiple interfaces (e.g., public class MyClass implements InterfaceA, InterfaceB).

3.2 Source Code Listings

Listing 3.1: Interface Definition (Relatable.java)

```
public interface Relatable {
    boolean equals(GradedActivity g);
    boolean isGreater(GradedActivity g);
    boolean isLess(GradedActivity g);
}
```

Listing 3.2: Implementing an Interface & Inheritance (FinalExam3.java)

```
public class FinalExam3 extends GradedActivity implements Relatable {
    private int numQuestions;
    private double pointsEach;
    private int numMissed;

    public FinalExam3(int questions, int missed) {
        this.numQuestions = questions;
        this.numMissed = missed;
        this.pointsEach = 100.0 / questions;
        setScore(100.0 - (missed * pointsEach));
    }

    @Override
    public boolean equals(GradedActivity g) {
        return this.getScore() == g.getScore();
    }

    @Override
    public boolean isGreater(GradedActivity g) {
        return this.getScore() > g.getScore();
    }

    @Override
    public boolean isLess(GradedActivity g) {
        return this.getScore() < g.getScore();
```

```
    }
}
```

Listing 3.3: Interface Polymorphism (PolymorphicInterfaceDemo.java)

```
public interface RetailItem {
    double getRetailPrice();
}

public class CompactDisc implements RetailItem {
    private String title;
    private double price;

    public CompactDisc(String t, double p) {
        title = t;
        price = p;
    }

    @Override
    public double getRetailPrice() {
        return price;
    }
}

public class PolymorphicDemo {
    // Method accepts ANY object that implements RetailItem!
    public static void showPrice(RetailItem item) {
        System.out.printf("Retail Price: $%,.2f\n", item.getRetailPrice());
    }
}
```

3.3 Deliverables for Lab Activity 3

Submit the following deliverables for Lab Activity 3:

1. **Source Code Files:**
    - Relatable.java (Interface specification)
    - FinalExam3.java (Class implementing Relatable while extending GradedActivity)
    - RetailItem.java (Interface specification)
    - CompactDisc.java (Class implementing RetailItem)
    - PolymorphicDemo.java (Driver demonstrating interface polymorphism)
2. **Comparative Execution Output:**

- Console output demonstrating comparison operations (equal, greater than, less than) between two FinalExam3 objects.
- Console output verifying that showPrice(RetailItem item) accepts diverse objects implementing RetailItem.

SUMMARY COMPARISON: INHERITANCE vs. ABSTRACT CLASSES vs. INTERFACES

Feature	Concrete Superclass	Abstract Class	Interface
Keyword	extends	abstract class ... extends	interface ... implements
Multiple Hierarchy?	No (Single Inheritance Only)	No (Single Inheritance Only)	Yes (Multiple Interfaces)
Direct Instantiation?	Yes (new Class())	No (Compiler Error)	No (Compiler Error)
Method Bodies	All methods must have bodies	Can have concrete & abstract methods	Abstract by default (unless default)
Fields	Any access modifier / instance fields	Any access modifier / instance fields	Implicitly public static final
Primary Purpose	Code reuse & specialization	Partial framework contract	Pure behavioral specification

LAB ASSESSMENT & GRADING RUBRIC

Evaluation Criteria	Weight	Exemplary (90–100%)	Developing (<70%)
OOP Architecture & Design	35%	Proper extends / implements usage, correct abstract methods, 100% encapsulation.	Missing annotations, direct field access, flawed inheritance structure.
Code Correctness & Execution	35%	Compiles without errors or warnings. Produces accurate output.	Compilation errors, logic flaws in calculations or method bindings.
Polymorphic Implementation	20%	Demonstrates dynamic binding via superclass & interface reference arrays.	Monolithic code without polymorphic array traversal.
Lab Documentation & Answers	10%	Clear answers to conceptual questions regarding constructors & abstract errors.	Incomplete or missing answers to conceptual questions.